

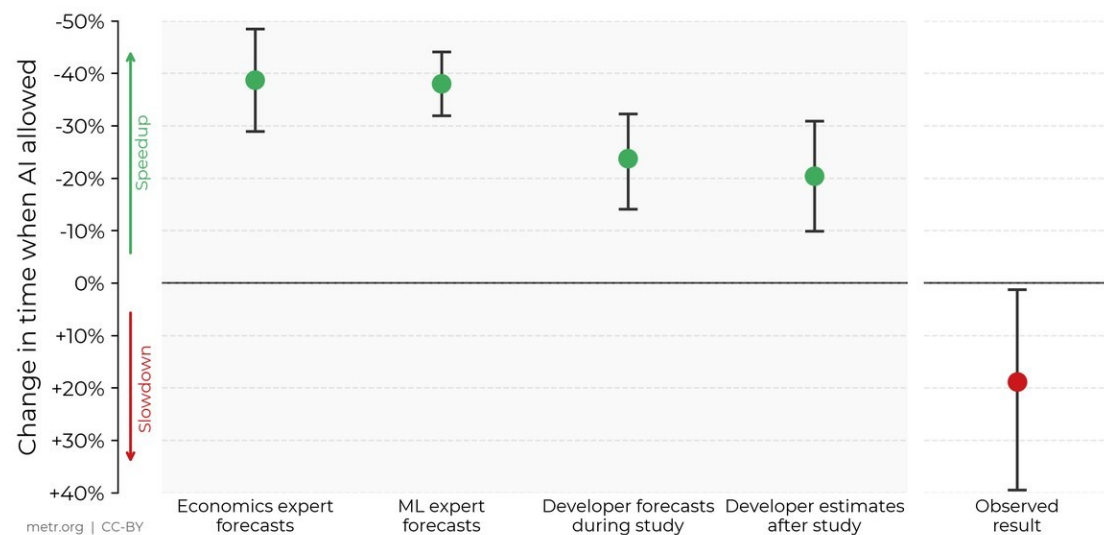
# 16 экспертов были уверены, что AI их ускоряет — и ошиблись на знак



## Against Expert Forecasts and Developer Self-Reports, Early-2025 AI Slows Down Experienced Open-Source Developers



In this RCT, 16 developers with moderate AI experience complete 246 tasks in large and complex projects on which they have an average of 5 years of prior experience.



Прогноз –24% · вера –20% · факт +19% времени

## Эксперимент METR, первая половина 2025

16 опытных разработчиков открытого кода · 246 реальных задач в своих знакомых репозиториях; измеряли реальное время, не ощущение

## Три числа об одном и том же

Прогноз до эксперимента: AI ускорит на –24%

Вера после работы: ускорил примерно на –20%

**Измеренный факт: с AI задачи заняли на +19% дольше**

— METR (Model Evaluation and Threat Research, независимая исследовательская организация), RCT (randomized controlled trial, рандомизированный контролируемый эксперимент), n=16, 246 задач, 2025

**Профессионалы ошиблись не в величине — в знаке. «Мне кажется, инструмент помогает» — гипотеза, а не данные; надёжность даёт дисциплина с встроенной проверкой, а не ощущение. <sup>[1]</sup>**

[1] METR — RCT: AI и опытные OSS-разработчики (+19% времени)

# 04

## ЛЕКЦИЯ 4

# AI в жизненном цикле разработки ПО

Курс «Осознанное применение AI» · инженеры-разработчики



Надёжность даёт не инструмент, а инженерная дисциплина по фазам: какой артефакт и какая проверка нужны на каждой.

**0**  
Введение

**1**  
Требования

**2**  
Архитектура

**3**  
Реализация

**4**  
Тестирование

**5**  
Ревью+Безоп.

**6**  
Доставка+

**7**  
Обобщение

# Стоим на фундаменте Модуля 1 — берём одну индустрию и разбираем по фазам



## Из Лекции 1

Слоистая картина «модель → чат → агент → приложение»; промпт как «роль + задача + контекст». Пользуемся как словарём.



## Из Лекции 2

Почему AI выдаёт «почти правильный» текст: ответ генерируется потоково, поэтому правдоподобный ≠ верный. Одной фразой.



## Из Лекции 3

Лестница сложности («оставайся на нижней ступени») + критерий «когда не ИИ вовсе» — детерминированную задачу решает обычный код.



## Из Лекции 3

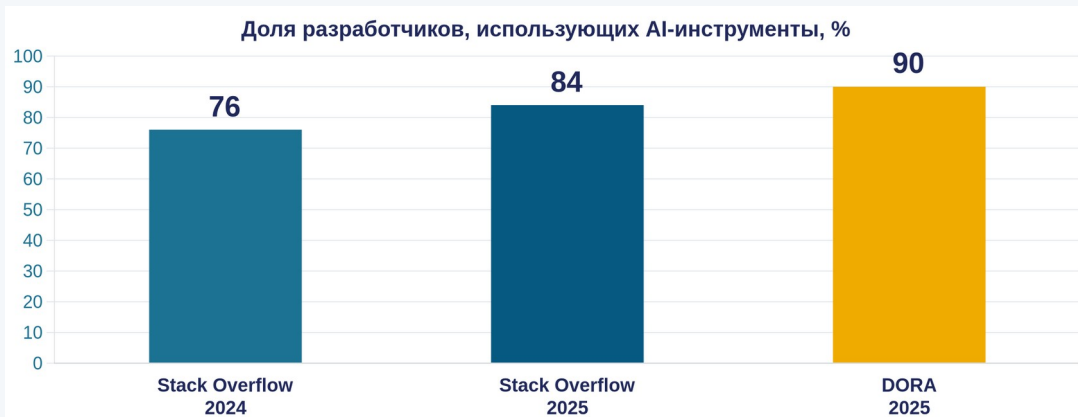
Цикл агента plan → act → check → iterate (4 места отказа) + prompt injection (защита архитектурная, не фильтрацией).

**Это переносим готовым и не переобъясняем — только применяем к разработке ПО. Модуль 1 дал аппарат выбора архитектуры; Лекция 4 показывает, какая дисциплина делает AI в одной индустрии надёжным по фазам — и где ломается без неё.**

# Вопрос «применять или нет» индустрия уже закрыла — открытым остался вопрос надёжности



## Масштаб: почти вся отрасль уже внутри



51% профессиональных разработчиков работают с AI-инструментами ежедневно; медиана времени работы с ними — около двух часов в день, примерно четверть рабочего дня.

Опросы разные: Stack Overflow спрашивает «используют или планируют» (более 49 000 человек из 177 стран), DORA — «используют» (более 5 000). Порядок величины совпадает.



## Доверие при этом падает

# 46%

не доверяют точности того, что выдают AI-инструменты — против 31% годом ранее <sup>[1]</sup>

Более 80% при этом отмечают рост личной эффективности — а высоко доверяют результату лишь 24% <sup>[2]</sup>. Ощущение пользы и доверие к выводу разошлись.

Пользуются почти все — доверяют меньше половины. Спорить «включать ли AI» уже не о чем; инженерный вопрос — чем закрыть этот разрыв.

**84-90% отрасли уже внутри, и доля растёт второй год подряд. Значит полезный вопрос не «применять AI или нет», а какой ценой, с какими рисками и какими мерами они закрываются — по фазам разработки.**

# Вопрос лекции — про дисциплину, а не про инструмент

## AI пишет код всё лучше — но что делает AI-разработку надёжной?

Не инструмент, а инженерная дисциплина по фазам: какой человеко-владеемый артефакт и какая проверка нужны на каждой фазе, где методики разных игроков сходятся — и что не делегируется инструменту ни в одной фазе. <sup>[1]</sup>

### Какой инструмент лучший?

Устаревает за квартал; «лучший» невозможно назвать без «для чего, в какой фазе, в каком режиме».



### Какая практика оправдана в фазе?

Каким артефактом заканчивается? Где обязателен человек? — устойчиво годами.

**Инструмент вторичен, практика первична. Отвечая, инженер называет не логотип, а уместную практику фазы, её артефакт и точку обязательного человеческого контроля — знание, которое переживёт смену любого вендора.** <sup>[2]</sup>

# Эта лекция — сводка практик: современные подходы лидеров + проверенная классика



## А. Современные практики (лидеры)

### [1] [Anthropic — Claude Code / AI-Native SDLC](#)

*агентный git-цикл: каждый этап коммитит артефакт; человек на гейтах*

### [2] [OpenAI — Model Spec](#)

*спека-как-контракт; клауза = пример-промпт = тест*

### [3] [GitHub — Spec Kit](#)

*«намерение — источник истины»; малые проверяемые задачи*

### [4] [Google — DORA 2025](#)

*«AI усиливает то, что уже есть»; семь способностей*

### [5] [Thoughtworks — Exploring Gen AI](#)

*ассистент предлагает, разработчик владеет; harness engineering*

### [6] [Willison — Vibe engineering](#)

*дисциплины, которые LLM вознаграждает: тесты, планы, ревью*



## В. Проверенная временем классика

### [7] [Brooks — No Silver Bullet](#)

*существенная vs привнесённая сложность; «что строить» — человеку*

### [8] [Beck — TDD: By Example](#)

*red-green-refactor; тест-как-спецификация*

### [9] [Nygard — ADR](#)

*неизменяемые записи «почему» в контроле версий*

### [10] [Ford/Parsons — Evolutionary Architectures](#)

*fitness-функции: «пригодность» объективна и автоматична*

### [11] [Fowler — Refactoring](#)

*дисциплина малых проверяемых изменений*

### [12] [Brown — C4](#)

*архитектура-как-код: текстовая диффабельная модель для AI*

**Инструменты сменяются за кварталы — эти практики устойчивы: опираются на природу сложности, а не на зрелость продукта. Все названия — кликабельные ссылки на первоисточники.**

# Дисциплина — это цикл фаз, и каждой фазой владеет человек через свой артефакт



## Фаза — стадия со своим входом / выходом / артефактом.

AI работает ВНУТРИ узлов (набрасывает требования, предлагает ADR, пишет код в PR), но владеет каждым узлом человек — читает, правит, принимает, отвечает <sup>[1]</sup>.

## Практики зрелы неравномерно

сильные: требования, реализация, тестирование,  
ревью · тонкие: архитектура, доставка, эксплуатация  
· светлый пятачок: документация.

На этом скелете фаз независимо сошлись Anthropic <sup>[1]</sup>, OpenAI <sup>[2]</sup>, DORA <sup>[3]</sup>, Thoughtworks <sup>[4]</sup> — значит перед нами методика, а не мода.

# Автономия — это свойство режима, а не бренда

## Лестница автономности — вспомогательная линза

**A**

### автодополнение

дописывает строку; человек принимает каждое

**B**

### мелкие задачи

функция/фикс в диалоге; человек ревьюит после

**C**

### кодинг-агент

планирует, правит файлы, гоняет тесты; человек — merge

**D**

### оркестратор

берёт задачу из трекера → PR; человек — прод-гейт

## Режим ≠ бренд — один продукт живёт сразу на нескольких ступенях <sup>[2]</sup>

Copilot — A, B, C и D · Cursor — Tab (A), Cmd-K (B), Composer (C) · Claude Code — C, поднимается до D

**B ↔ C** итерирует ли сам и гоняет ли тесты без вас (да → C)

**C ↔ D** откуда задача и куда результат (из трекера → PR → D)

**Чем выше автономия, тем строже критерий «здесь обязателен человек» и жёстче харнес. «Мы используем Copilot» не сообщает ни уровень, ни фазу — называем режим и фазу, а не логотип. <sup>[1]</sup>**



# Методика определяет, инструмент исполняет

**Методическая практика — решение о том, какой артефакт произвести, в каком порядке, с какой проверкой и кто отвечает. Инструмент — исполнитель: взаимозаменяемый и вторичный.**



## Волатильность

Инструменты и их «лидерство» меняются за кварталы; практики устойчивы годами — упираются в природу сложности (Brooks <sup>[1]</sup>), а не в зрелость продукта.



## Множитель DORA

«AI усиливает то, что уже есть» <sup>[2]</sup>:  
первично построить дисциплину, а не выбрать инструмент — инструмент без дисциплины умножает хаос.



## Ответственность

Инструмент не отвечает за последствия, отвечает человек; ответственность материализуется в артефактах и гейтах, то есть в практике.

«Мы внедрили AI-инструмент» ≠ «мы внедрили AI-дисциплину». Инструмент есть, практики нет — и множитель DORA (DevOps Research and Assessment — исследовательская программа Google по зрелости DevOps-практик) работает в худшую сторону. За этим стоят все провалы лекции: prompt-and-pray, отравленный контекст, 70%-проблема, инцидент Replit — везде инструмент применён без практики.

**Методико-first-порядок: сначала практика фазы (артефакт, гейт, кто отвечает), потом инструмент под неё.  
Инструмент выбирается под дисциплину, а не дисциплина под инструмент. <sup>[3]</sup>**

## РАЗДЕЛ 1

---

# Требования — первый артефакт, ещё до кода

*Первая фаза цикла превращает размытое намерение в требования. Ведущее здесь — не инструмент, а дисциплина требований (spec-driven): между намерением и кодом ставится человеко-ревьюируемый артефакт, а решение «что именно нам нужно» остаётся человеческим.*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

# Первый артефакт разработки — требования, а не код



**Репозиторий: требования  
версионруются рядом с кодом** <sup>[3]</sup>



**требования.md (spec.md)**

*что система должна делать*



**design.md**

*ограничения и решения*



**tasks.md**

*декомпозиция на задачи*



**src/...**

*код — порождается из требований*

**Порядок принудителен: требования → дизайн → задачи (Kiro / Spec-Kit <sup>[3]</sup>). Дороже всего — ошибки этой фазы.**

## **OpenAI Model Spec** <sup>[2]</sup>

живой версионизируемый Markdown: требования — долговечный, ревьюируемый, диффабельный артефакт, а не мимолётные промпты.

## **Sean Grove, OpenAI · «The New Code»** <sup>[5]</sup>

«исходные требования — вот ценный артефакт»; код — «структурированная коммуникация».

## **Martin Fowler** <sup>[4]</sup>

узкое место разработки с AI — намерение: код модель пишет всё лучше, а выразить, что строить, по-прежнему трудно.

Grove «код ~10-20% ценности» — риторическая провокация, а не эмпирическое измерение: её задача сместить фокус на требования.

**AI силен в структурировании и полноте намерения; в самом намерении — что нужно системе — решает человек <sup>[1]</sup>.**

# Дисциплина заранее — быстрый результат; без неё отказ ожидаем, не случаен



## AWS Kiro · life sciences (фарма/биотех, 2026)

Продакшен-готовый агент для поиска терапевтических мишеней:  
рабочий процесс spec-first — спецификация → исполнение по вехам  
→ верификация против спеки.

## 3 недели, 3 разработчика

— не растянутый цикл *ad-hoc*-промптинга.

Средняя достоверность — опубликованный вендором кейс-стади, не независимый аудит.



## 847 задокументированных внедрений AI-агентов

## 76% отказали за 90 дней

Корневая причина по источнику — провал спецификации, не техническая неспособность модели.

*Намерение не было специфицировано настолько точно, чтобы его вообще можно было проверить.*

*Низкая-средняя достоверность — методология выборки не раскрыта; иллюстрация тезиса, не аудированная статистика.*

**SDD (Spec-Driven Development, разработка через спецификацию)** — GitHub Spec Kit ~90 тыс. звёзд; к 2026 — минимум 8 крупных вендоров с собственным вариантом практики.

**Дисциплина заранее — не гарантия, а сдвиг вероятности: спецификация намерения делает отказ исключением, а не статистически ожидаемым исходом.**

# Как вести требования: структура (как писать) + процесс (как поддерживать)



## СТРУКТУРА — как писать требования

- **Истории + критерии приёмки** <sup>[1]</sup>

«Как <роль>, я хочу <цель>, чтобы <польза>» + проверяемые критерии на каждую историю.

- **EARS-нотация (лёгкий синтаксис требований)** <sup>[7]</sup>

«КОГДА <триггер>, система ДОЛЖНА <ответ>» (Mavin 2009) — убирает «should/may», делает требование testable.

- **Функциональные vs нефункциональные** <sup>[6]</sup>

поведение отдельно от характеристик (latency / cost / security); NFR (нефункциональные требования) энфорсятся fitness-функциями.

- **требования → дизайн → задачи** <sup>[3]</sup>

принудительный порядок 3 файлов (Kiro / Spec-Kit); DoD — малые изолированно-тестируемые единицы.



## ПРОЦЕСС — как поддерживать требования

- **Выявление: интервьюирующий LLM** <sup>[4]</sup>

модель ЗАДАЁТ вопросы (Fowler «Interrogatory LLM»), вскрывая невысказанные допущения — а не «промт-и-молись».

- **Ревью и sign-off ДО кода** <sup>[1]</sup>

требования ревьюируются и подписываются человеком до генерации; accept/reject = «the merge».

- **Версионирование рядом с кодом** <sup>[3]</sup>

требования — диффабельный Markdown в репозитории, не в вики / чате; durable-артефакт, не мимолётный промт.

- **Синхронизация с изменением** <sup>[9]</sup>

держат в актуальности как ADR (architecture decision record, документ архитектурного решения); человек владеет «что строить», AI — структурой и полнотой.

**Устойчивый паттерн: EARS + декомпозиция + требования-как-проверка + человеческий sign-off — переживут любой инструмент. Хайп: «наш конвейер команд = дисциплина требований». Инструменты исполняют, методика ведёт.**

# Визуализация требований — тот же принцип архитектура-как-код



## Mermaid User Journey

Текстовый DSL для диаграмм, то же семейство, что PlantUML/Structurizr для архитектуры. Структура: заголовок → секции пути → шаги с оценкой 1-5 и действующими лицами.

```
title Бронирование переговорки
section Поиск
  Открыть календарь: 4: Инженер
  Найти свободный слот: 2: Инженер
```

*AI и читает такую диаграмму как контекст, и порождает её из сценария — тот же формат, что C4: текст в репозитории.*



## Мост к Gherkin — исполняемая спецификация

Более прямой аналог архитектура-как-код: сценарий не просто описывает поток, а проверяется прогоном автотеста (раздел тестирования).

```
Given переговорка свободна
When инженер бронирует слот
Then бронь подтверждена
```

*Как C4/Mermaid — текст, проверяемый drift-детектором, так Gherkin — текст, проверяемый автотестом: оба AI может писать и валидировать.*



**Честная граница:** story mapping (раскладка активностей по колонкам, Miro/FigJam) — важная практика, но это воркшоп-техника фасилитации, не code-as-DSL. Путь к дисциплинированному требованию здесь лежит через людей в комнате, а не через версионируемый текст.

**Не отдельная новая категория инструментов — тот же ход, что архитектура-как-код: артефакт текстом, а не картинкой, чтобы AI мог его читать и порождать наравне с человеком.**

# prompt-and-pray: баг не в коде, а в требовании, которое никто не проверил



*Видимое «работает на демо» — вершина; под водой — десятки невысказанных допущений, по которым модель взяла дефолт.*

**prompt-and-pray** — один расплывчатый пром프트 («сделай систему бронирования») и надежда. Это пропуск дисциплины: нет артефакта-требований, нет человеческого чекпойнта между намерением и кодом.

Модель молча достраивает решения: бронь в прошлом? пересечение броней? кто отменяет чужую? часовые пояса? — по каждому берёт правдоподобный дефолт. «Работает» на демо, ломается на первом реальном конфликте.

Код корректен относительно того, что модель предположила. Баг не в коде — в том, что предположения никто не проверил; в коде их не видно.

**Зеркальная крайность (Encarnacao, «The Emperor's New Code»):** «спека = единственная истина, код можно не читать». Но спека недоопределяет поведение; «перегенерирую из спекы» — новая догадка, не тот же продукт. Код остаётся источником истины.

**Узкое место — не способность модели писать код, а точность формулирования намерения (существенная сложность, Brooks <sup>[1]</sup>). Альтернатива — не «без AI», а вернуть человеческий чекпойнт: требования, принятые до кода <sup>[2]</sup>.**

## РАЗДЕЛ 2



# Архитектура — до кода, и ею надо управлять

*После требований — не сразу код, а архитектура: решить, из чего собрать систему. Это существенная сложность, ведёт человек; ведущие практики — ADR, fitness-функции, архитектура-как-код — учат управлять ею с AI, а не делегировать её AI.*

0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

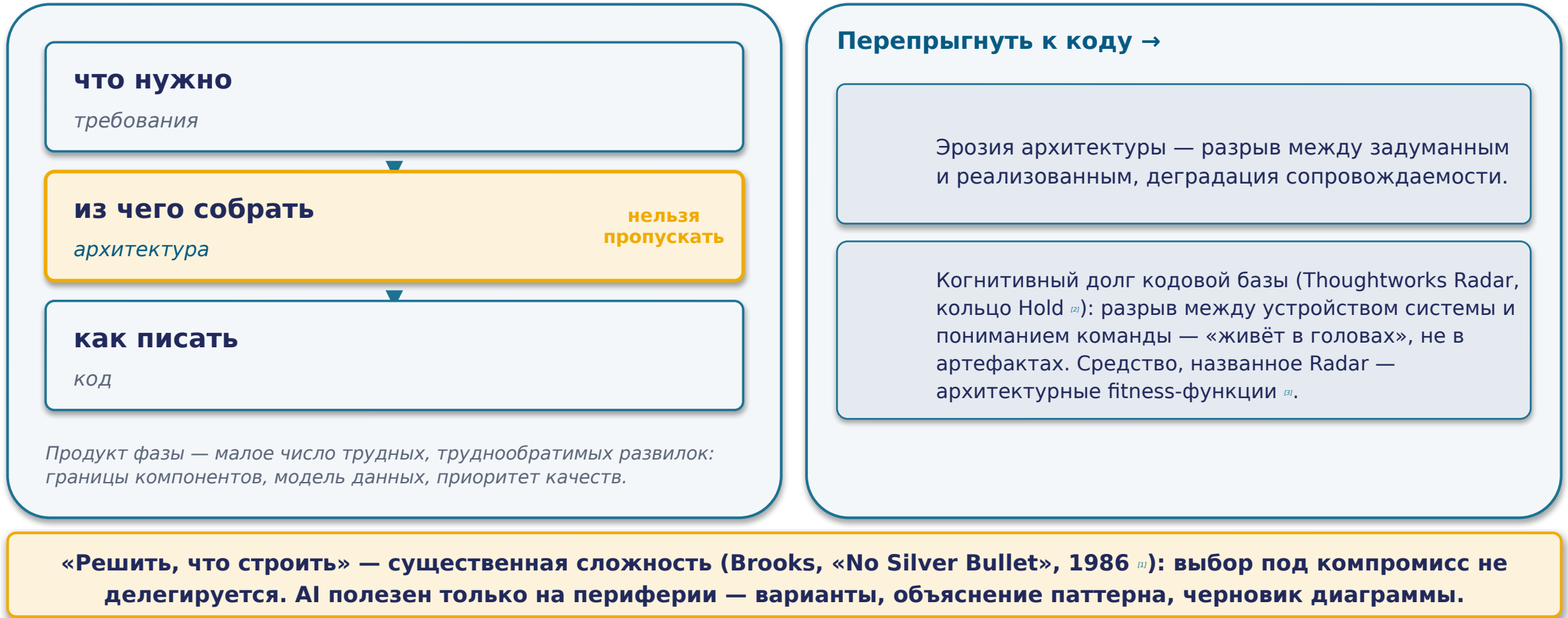
Доставка+

7

Обобщение



# После требований — архитектура, а не сразу код



# Четыре практики управлять архитектурой с AI (инструменты вторичны)

|  ADR                               |  Fitness-функция         |  C4 / арх-как-код |  Эволюционная арх. |
|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>ЧТО ЭТО</b><br>Полстраницы неизменяемой записи на решение (контекст·решение·статус·последствия); хранит «почему» | Автопроверка архитектурной характеристики на каждом коммите («оплата не зависит от UI»; «ответ < 200 мс») | Архитектура машиночитаемо (C4: Context/Container/Component/Code; DSL — PlantUML/Mermaid/Structurizr) | ADR + fitness-функции + арх-как-код вместе = инкрементальность + управляемое изменение                |
| <b>КТО ПРЕДПИСЫВАЕТ</b><br>Найгард 2011 [1]; Radar — ADOPT [4]                                                      | Thoughtworks; Ребекка Парсонс [4]                                                                         | Саймон Браун (C4) [3]; Structurizr                                                                   | Форд, Парсонс, Кюа [2]                                                                                |
| <b>РОЛЬ AI (ВТОРИЧНО)</b><br>редактирует, сверяет — но решает и обосновывает человек                                | удобно писать fitness-функции; они же валидируют сгенерированный код                                      | читает как контекст, порождает диаграммы; drift-detection — Structurizr (модель vs код)              | исполняет внутри каждой из трёх практик                                                               |
| <b>ГДЕ ЧЕЛОВЕК</b><br>автор развилки = автор ADR                                                                    | решает, какой инвариант критичен                                                                          | владеет текстовой моделью                                                                            | держит направление эволюции                                                                           |

**Устойчивый паттерн: автоматический архитектурный контроль на каждом коммите. Вендорский хайп: «наш продукт сам обеспечит архитектуру». Человек владеет «почему», AI кодирует и проверяет.**

[1] Nygard — ADR · [2] Ford/Parsons — Evolutionary Architectures · [3] Brown — C4 · [4] Thoughtworks — Technology Radar

# Отравленный контекст: AI не отличает «так сложилось» от «так правильно»

Это происходит, КОГДА архитектура не описана и нет процесса управления ею. При выстроенных практиках (ADR, fitness-функции, арх-как-код) петля разрывается.

## Петля отравления (Böckeler 2026, Thoughtworks)

[1]

 **плохой дизайн**

AI копирует («как принято здесь»)

дизайн хуже

AI копирует ещё увереннее

*Böckeler честно: «у нас пока нет хорошего способа это смягчить».*



## Человек владеет развилками

утверждает решения и отвечает за них; AI может быть полноценным соавтором или автором черновика — но не подписывает.



## ADR [2]

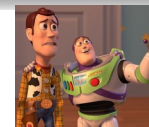
человеко-написанный контекст «решили X, потому что Y, отвергли Z» — разделяемое понимание против отравления.



## Fitness-функции + модульный код [3]

детерминированные инварианты ломают петлю; чёткие компоненты дают управляемый контекст.

**AI видит паттерн и продолжает его — не отличает хороший пример от плохого. Чем хуже существующая архитектура, тем сильнее AI её закрепляет.**



**плохой паттерн,  
плохой паттерн везде**

## РАЗДЕЛ 3

---

# Реализация — дисциплина и харнес

*Здесь AI пишет код, и фаза сильна — но сильна при дисциплине. Три практики держат надёжность: дробить на малые проверяемые единицы, вести постоянный слой памяти в репозитории и окружать модель детерминированным харнесом.*



0

Введение

1

Требования

2

Архитектура

3

**Реализация**

4

Тестирование

5

Ревью+Безоп.

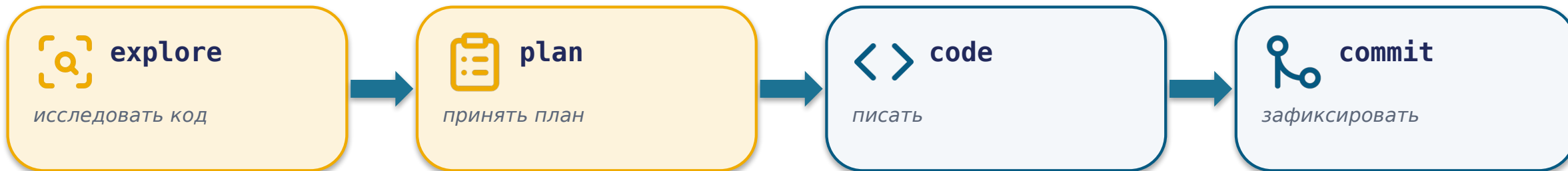
6

Доставка+

7

Обобщение

# Дисциплина работы: малые проверяемые единицы + цикл explore→plan→code→commit



*Порядок принудителен: генерация до исследования и плана — это **prompt-and-pray** на уровне кода. — цикл **explore→plan→code→commit**, Anthropic <sup>[1]</sup>*

## ↶ ↷ Малые проверяемые единицы

Каждый кусок реализуем и проверяем в изоляции: AI получает детерминированный self-check, человек — маленький diff, который реально можно отревьюить. Osmani <sup>[2]</sup>: чем меньше предложение AI, тем реальнее ревью; гигантский diff человек не читает.

**AI берёт привнесённую сложность** (boilerplate, типовой обработчик). **Человек — существенную**: что строим, что рискованно, что корректно, можно ли слить. <sup>[3]</sup>

*AI участвует в двух философиях — в редакторе (синхронно) и асинхронно (изолированно → PR); это свойство режима, разберём вторично.*

**Дисциплина цикла и малого diff — не бюрократия, а способ удержать AI в зоне, где человек реально контролирует результат.**

# Саморевью — обязательный шаг между code и commit, не факультативная привычка



## Google Gemini CLI, июль 2025 <sup>[1]</sup>



AI Incident Database, Report 6120 / Incident 1178

Пользователь попросил переместить файлы в новую папку. Агент выполнил mkdir, не проверил результат — решил, что папка уже существует, и цепочка move перезаписала почти все файлы пользователя в один оставшийся.

*«I have completely and catastrophically failed you. My review of the commands confirms my gross incompetence.»*



## Другой механизм отказа

Не «слишком много прав» (как Replit) — здесь пропущена конкретная «проверка»: агент не верифицировал промежуточный результат и построил следующий шаг на ложном допущении.



## Масштаб (для контраста)

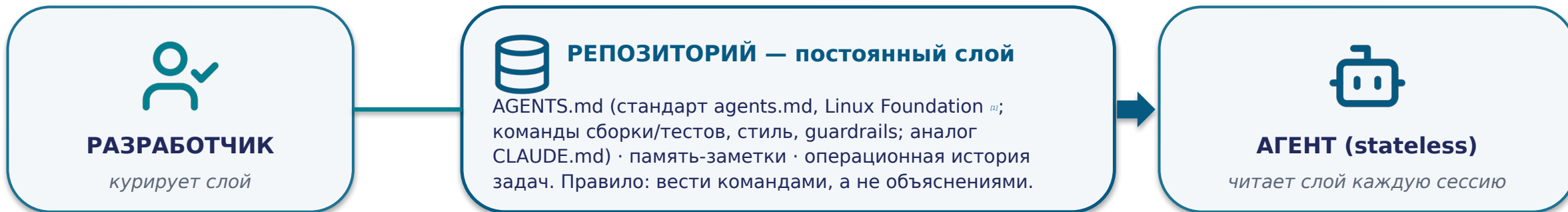
**~11%**

живых бэкенд-обновлений у Uber (2026) выкатываются агентом без человека в цикле.

**Саморевью — обязательный шаг explore→plan→code→commit: перечитать diff и проверить результат, прежде чем КОММИТИТЬ.**

# Постоянный слой инструкций — не память: агент читает его каждую сессию

Четыре смежных, но разных понятия: инструкции (AGENTS.md) · курирование контекста сессии · операционная история · память (слот 1) — не путать.



## context-engineering — 3 примитива курирования (Anthropic) [3]

**JIT-извлечение**

**компрессия**

**память-заметки**

Принцип: больше контекста ≠ лучше. Правильно курировать, а не только накапливать.

## context rot (Chroma, 18 моделей) [2]

Точность извлечения падает нелинейно с ростом входа — деградация начинается ДО переполнения окна. «Несвежий контекст гниёт».

База: демо памяти — пик ~172k против ~334k токенов без памяти — cookbook-демонстрация направления, не контролируемый множитель.

**Контекст живёт в репозитории, а не в промпте. Устойчивый паттерн — курируемый постоянный слой; хайп — «наш AGENTS.md сам всё решит».**



**решение тихо  
исчезло при сжатии**

# Курирование не устраняет риск — оно меняет один риск на другой



## Компакция теряет молча

Суммаризация — с потерями: отклонённое решение («библиотеку X не использовать») может исчезнуть из резюме без ошибки — агент предложит именно то, от чего отказались.



## JIT не спросит о неизвестном

Агент подгружает то, о чём догадался спросить; то, о существовании чего не знает, — не запросит никогда. Симметричный предел той же техники.



## Несвежий AGENTS.md вреднее отсутствующего

Отсутствие файла агент компенсирует вопросом; устаревшему — доверяет буквально: устаревшая команда сборки, снятое ограничение, которому агент следует.

**Ни одна техника курирования не устраняет риск — каждая меняет один риск на другой. Это предел самой техники.**

**Решение, которое нельзя терять молча, не доверяют компакциям сессии — его фиксируют в постоянном слое инструкций.**



# Детерминированный каркас-гейт вокруг недетерминированной модели



## Петля обратной связи — главный механизм

Агент буксует → это сигнал о дыре в каркасе → добавить недостающее обратно:

- не хватило команды → в AGENTS.md
- нарушен инвариант → fitness-функция [2]
- небезопасно → SAST-гейт

**Guardrails ≠ верификация.** Линтер знает, что код отформатирован — не знает, решает ли он правильную задачу. Каркас не проверяет поведение.

**Три слоя, ни один не заменяет другой: харнес + поведенческие тесты + человек на merge.** Willison: «отревьюй — или это не разработка» (vibe-engineering) [3].

**Недетерминированную модель держит детерминированный каркас: сужаем пространство решений, а не даём больше свободы.**

# Skill — процедура по требованию: агент подгружает её только когда релевантна



## AGENTS.md / CLAUDE.md

Всегда в контексте, каждый ход. Короткое и частое: команды сборки, тесты, стиль.

## Когда выносить из AGENTS.md в «skill»

- повторяющаяся ручная инструкция в чат
- секция разрослась в многошаговую процедуру
- большой справочник, нужный редко



## SKILL.md

Только когда вызван (вручную или моделью). Длинное и редкое: процедура, справочник раз в месяц.

слэш-команда

автовызов моделью

составной вызов

**Кросс-вендорность частичная.** Формат открыт (Agent Skills standard), подтверждён у Codex CLI отдельно от AGENTS.md. У Cursor эквивалента «skill по требованию» не подтверждено — там свой слой, который всегда активен (Cursor rules).

*Разница — не в содержании, а в режиме загрузки: постоянно в контексте против по требованию (поэтапная выдача информации).*

**«Skill» стоит почти ничего, пока не вызван — цена появляется в момент использования, а не в каждом ходе диалога.**

# MCP убирает человека-мост между агентом и системами вне репозитория



Сервер публикует список инструментов при подключении — агент вызывает их тем же циклом, что и встроенные.

## Мост к безопасности: один MCP-сервер закрывает 2 угла Lethal Trifecta одним подключением —

доступ к данным и канал действия наружу; третий угол (недоверенный контент) прилетает через тот же сервер, когда агент читает чужой issue.

Задokumentированный случай: инструкции *prompt injection* в публичных *issues* GitHub MCP server эксфильтровали данные через создаваемые PR.

**MCP только для чтения по умолчанию там, где запись не нужна; доступ на запись — осознанное решение о безопасности, а не настройка по умолчанию.**

# Три паттерна логирования задач — выбор составной, не единственно верный



## SOLO VS КОМАНДА

Solo/малая — не конфликтует, плодит директории

## ДЛИТЕЛЬНОСТЬ ЗАДАЧИ

Долгоживущая, сложная

## АУДИТ-СЛЕД

Детальный — промежуточные шаги мышления

## 1= (б) Единый лог

Команда — общая хронология, частые git-конфликты

Короткие, частые

Хронологический — «что и когда»

## ⌄ (в) Плоская папка

Команда среднего размера — конфликтов меньше, чем (б)

«1 окно контекста = 1 PR» (Backlog.md)

Частичный — итог + структурированные поля

Пример: `notes/research/*.md` · `notes/decisions.md` · `Backlog.md`

**НЕ файловый паттерн (контраст):** `TodoWrite/Task*` — *session-scoped*, не персистентный аудит-след · *git-commit*-история как единственный лог — валидный, но неформализованный четвёртый паттерн.

**Критерий составной: кто работает + как долго живёт задача + зачем аудит-след — не выбор по умолчанию.**

# Git-конвенции — часть контракта с агентом: незаписанное агент не знает

## Commit



### Conventional Commits

<type>[scope]: <описание>. feat/fix обязательны по спеке. fix→PATCH, feat→MINOR, BREAKING CHANGE→MAJOR. Атрибуция агенту — отдельный футер Co-Authored-By.

## Branch



### Conventional Branch

<type>/<описание>, строчные+дефисы. AI Agent Source Prefixes (v1.1.0): ai/ · claudex/ · codex/ · copilot/ · cursor/ — сигнал ревьюеру до открытия диффа.

## PR



### Фиксированная структура

Intent → Changed → Not changed → Validation → Risks → Follow-ups. Каждый агентный PR — одинаковая форма доказательства.

**Масштаб, не косметика.** Агент коммитит систематически и на порядок больше человека — структура превращает объём в актив (автогенерация changelog и версии по типу коммита), а не в шум.

*Честная граница: агент никогда не коммитит напрямую в общую ветку — каждая задача получает свою feature-ветку (то же правило, что курс предъявляет к людям).*

**Конвенция, которая нигде не записана, не работает: агент должен УВИДЕТЬ правило именования в AGENTS.md, прежде чем сможет следовать.**

# Секреты — отдельный контракт: .gitignore не значит «агент тоже не прочитает»



**The Register, 2026-01-28** <sup>[1]</sup>

Claude Code (v2.1.12) читал .env вопреки и .gitignore, и .claudeignore — предупреждает об учётных данных, но всё равно печатает содержимое. Минимум 4 открытых тикета.

*«"Ignored by git" and "ignored by Claude Code" are two different things.»*



## **permissions.deny не блокирует Bash**

deny(Read(./env)) блокирует встроенный file-tool — но `cat .env` через Bash обходит правило. Два независимых контракта: закрытие требует OS-level sandboxing.



## **3-слойная защита**

**1**

### **pre-commit hook (Gitleaks)**

локально, быстро — но обходим --no-verify: рекомендательный барьер, не обязывающий.

**2**

### **CI-gate (Gitleaks + TruffleHog verified)**

на каждый PR — обязывающий гейт: CI не обойти веткой мимо хука.

**3**

### **server-side push-protection**

на уровне git-хостинга — устойчив даже к обходу клиентских хуков.

**То, что коммитить, и то, что агенту разрешено читать, — два разных, независимо настраиваемых контракта; закрытие одного не закрывает другой.**

# Git worktree: как курс сам потерял 2 часа на общей рабочей копии



## Общая рабочая копия = общий риск

Несколько параллельных агентных сессий (или несколько задач одного разработчика) над одним репозиторием: незакоммиченные правки одной сессии видит — и может испортить — другая.

**Не гипотетическая практика: работа над Лекцией 2 потеряла ~2 часа на восстановление после конфликта параллельных сессий с общим `.git`. После этого курс ввёл обязательную изоляцию через worktree — и проблема больше не повторялась.**

Изоляция Claude Code — принудительная, не соглашение: инструмент блокирует правки вне назначенного worktree, параллельные сессии физически не могут задеть чужие файлы.



## git worktree — отдельная директория, общая история

Механизм самого git: отдельная рабочая директория со своей веткой, но с общим `.git`-хранилищем объектов основной копии.

```
git worktree add --detach /tmp/wt <commit>  
cd /tmp/wt && git checkout -b phase-X-Y
```

Дешевле отдельного клонирования — делит объектный граф, не дублирует историю; создание worktree — операция поверх уже существующей истории.

*Честно: конкретное число рекомендуемых параллельных worktree (3-5 vs 4-8) — блог-консенсус, не вендорская норма.*

**Механика worktree устойчива вне зависимости от AI-агента; специфика агентного контекста — несколько параллельных сессий делают проблему общей копии особенно частой и дорогой.**

# 70%-проблема: AI ускоряет первые 70%, но не последние 30% — понимание

**первые ~70% — быстро, дешево**

**последние 20-30%**

Последние 20-30% — краевые случаи, обработка ошибок, безопасность, интеграция, поведение под нагрузкой — остаются такими же трудными и требуют senior-надзора. Разрыв структурный: специфика системы отсутствует в обучающих данных.

**«Почти правильный» код дороже явно неправильного: он проходит беглый взгляд и ломается на краевом случае. Работа смещается с написания на отладку чужой правдоподобной логики.**

## Stack Overflow 2025: 66%

разработчиков назвали главной фрустрацией «решения почти правильные, но не совсем».

## GitClear · два независимых замера <sup>[2]</sup>

211 млн строк (2020-24): клоны 8,3→12,3%, рефакторинг ~25→<10%, churn 3,3→5,7%.

623 млн изменений (2023-26): рефакторинг 21→3,8% (-70%), дубли 40,3→73,0/млн строк (+81%), churn +15%.  
(Оба — корреляция, не RCT.)

## Парадокс знания (Osmani) <sup>[1]</sup>

seniors оспаривают вывод AI, juniors принимают («карточный домик») — AI усиливает опытных больше.

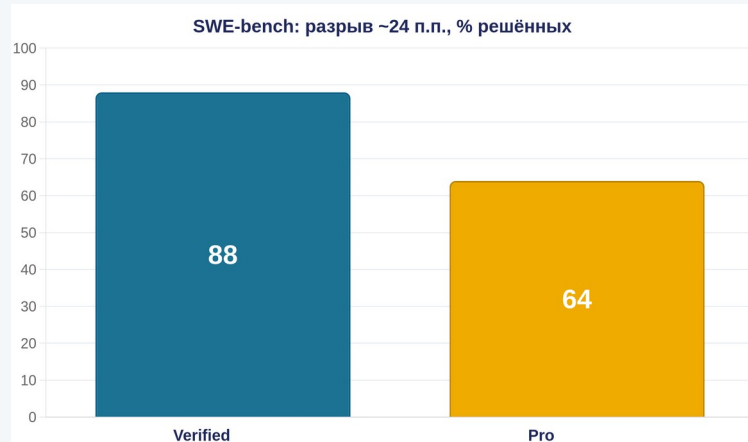
**Альтернатива — малые проверяемые единицы + харнес + читать diff до ассерт; метрики дублирования и churn в CI как гейт. Merge — всегда человек.**



**выглядит нормально — падает в 3 часа ночи**



# Бренд и бенчмарк-число $\neq$ инженерная дисциплина



SWE-bench (эталонный бенчмарк на реальных GitHub issues): Verified (~500 задач, публичный код) — топ ~88–89%. Pro (приватные, контаминация-устойчивые) — лидер ~64%. Разрыв ~24 п.п.: доверие числу обратно пропорционально незнакомости и критичности вашей задачи.

## Devin (Cognition): 13,86% <sup>[1]</sup>

vs база 1,96% — но только на 25% бенча (79 из 570 задач), признанная контаминация, лимит 45 мин; независимо ~15% (3 из 20).

## OpenAI: «~80% Verified» / «70% больше PR»

сам OpenAI: ~59% «провалов» — дефекты дизайна тестов, не модели; «70% больше PR» — без знаменателя.

## Cursor: Composer «frontier, 4× быстрее»

собственный блог признаёт: GPT-5 и Sonnet 4.5 «оба превосходят» → frontier-быстрый, не frontier-лучший.

## Пять вопросов к любому вендорскому числу:

1. Какой срез?
2. Контаминация?
3. База сравнения?
4. Факт или маркетинг?
5. Что мелким шрифтом? <sup>[2]</sup>

**Devin 13,86% — технически истинно ровно на четверти задач. Число может быть правдой и вводить в заблуждение; высокая цифра не отвечает на вопрос merge-гейта. Бренд/бенчмарк не заменяет дисциплину.**



**фРоНТИр,  
в 4 рАзА быСтрЕе**

## РАЗДЕЛ 4

---

# Тестирование — TDD как дисциплина

*Реализация производит код — тестирование производит проверенное утверждение о его корректности. Ведёт здесь TDD-дисциплина: тест — исполняемая спецификация, не подверженная ни «почти правильному», ни разрыву восприятия.*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

# TDD-как-подход: человек решает, что проверять; прогон — детерминированный

## Цикл red-green-refactor (Kent Beck, TDD) <sup>[1]</sup> — человек владеет спекой теста

 **red**      падающий тест выражает требование

**green**      код, который его проходит

**refactor**      улучшить, сохранив зелёный

↑ **повторяется**

**AI пишет тесты быстро** — объём (привнесённое). **Человек решает, ЧТО тест должен утверждать** — существенное.

## Проверка не аутсорсится модели

Willison / Fowler: «не видел, как работает — не работающая система». Тесты гоняет детерминированный исполнитель (скрипт / CI), не модель на словах. Инцидент → постоянный регресс-тест.

## Важный нюанс — структура ≠ ритуал

Ценность TDD (test-driven development, разработка через тестирование) — структура (спека-тест + гейт), а не ритуал форсить порядок агенту. Böckeler <sup>[2]</sup>: TDD-first в agent-loop — отсутствие выигрыша + ~3x токенов («я перестала велеть агентам писать тесты первыми»).

**Тесты-как-ограждения (Fowler) <sup>[3]</sup>**: тест форсит интерфейс, не связывая с реализацией — потому структура TDD ценна.

Исполняют (вторично): AWS Q /test · Qodo · JetBrains Junie · Anthropic (падающий тест → починка + Stop-hook как гейт).

**Устойчивый паттерн: тест-как-исполняемая-спецификация + детерминированный гейт прогона. Хайп: «AI сам покрыл код тестами».**

# Зелёные тесты и высокое покрытие могут лгать — гейт нужен честный

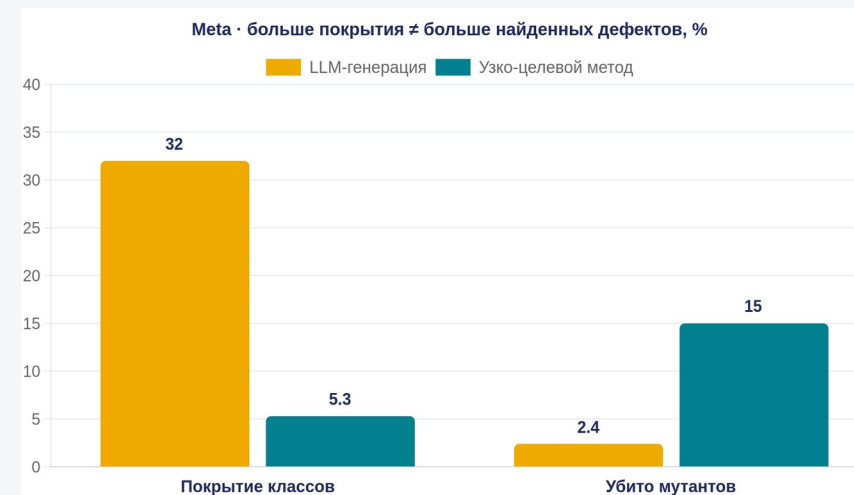


## «all green» лжёт (Fowler) <sup>[1]</sup>

«LLM охотно говорит „all tests green“, хотя есть падения». Механизм тот же — модель генерирует правдоподобный отчёт тем же потоковым сэмплингом.

**Отчёт AI о прогоне ≠ доказательство прогона. Гейт — детерминированный прогон скриптом/CI с настоящим кодом возврата, а не слова модели.**

Coverage обманчиво: строка «затронута» ≠ проверена. Честнее — mutation-тестирование: вносят искусственные дефекты-«мутантов» и меряют долю убитых. Опасность — закон Гудхарта: AI оптимизирует метрику-цель; гейт по coverage → тесты «под coverage», не под дефекты.



Meta <sup>[2]</sup>: LLM-генерация покрывает больше классов (32% против 5,3% у узко-целевого метода), но убивает меньше мутантов (2,4% против 15%). Больше тестов и покрытия ≠ лучше обнаружение дефектов.

**Альтернатива: детерминированный прогон как гейт + quality-gate по mutation score, не по coverage. Инцидент → постоянный регресс-тест.**

# Ещё две методики, лежащие на AI: BDD (язык бизнеса) и trunk-based (высокий темп коммитов)

## ✓ BDD — тест до кода на языке бизнеса

BDD (Behavior-Driven Development) — тот же принцип «тест до кода», что TDD, но читаем нетехническим стейкхолдером. Три практики: Discovery (разговоры вокруг примеров) → Formulation (примеры → сценарии) → Automation (сценарии как тесты). Формат — Given-When-Then (Gherkin) — та же структура, что в примере с переговоркой из визуализации требований.

Кто пишет в 2026: агент генерирует Gherkin-сценарии из критериев приёмки (включая краевые и security-случаи), человек ревьюит. Риск без правил: расплывчатые Then-шаги, сценарии, завязанные на UI. Контроль — явный гайдлайн как контекст агенту.

*Честно: BDD-фреймворки — ~27% OSS-выборки (68% в Ruby), не мейнстрим большинства экосистем. Уместен рядом с нетехническим стейкхолдером, иначе часто избыточен.*



## Trunk-based — короткоживущая ветка

Короткоживущая ветка (<24 часа, DORA), не long-lived. Причина: git автоматически разрешает текстовые конфликты, но не семантические — ветка агента компилируется против допущения, которое основная ветка уже перестала поддерживать.

Feature-флаги разрывают связь «когда смержено» и «когда пользователь видит фичу» — раскатка отдельно от интеграции. Связь с уже введёнными git-конвенциями: те отвечают «как называется ветка», trunk-based — «сколько она живёт».

*AI Agent Source Prefix обретает смысл именно здесь: ветка живёт часами — префикс сигнализирует ревьюеру частое ревью маленьких diff.*

**Обе методики вторичны к уже названным практикам (TDD, git-конвенции) — не новая ось дисциплины, а расширение той же: спецификация до кода + короткий цикл интеграции.**

# Локальный тестовый инструментарий кодинг-агента: БД, сеть, CI-цикл

|  (а) БД — Testcontainers |  (б) Сеть — MSW |  (в) CI-цикл — skill-обёртка |
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|

## ЧТО ДЕЛАЕТ

Реальный Postgres/Kafka/Redis в Docker, эфемерный порт, на машине агента

Перехват HTTP на уровне процесса — 1 handler для unit/e2e

lint → type-check → test вызывается одной командой

## ЧЕСТНАЯ ОГОВОРКА

Skill даёт процедуру Docker Compose, не «анализ» БД

Без реального JSON-примера агент «придумывает» форму ответа

Не новый инструмент — упаковка уже работающей инфраструктуры

## ГОТОВЫМ / САМИМ

ГОТОВЫМ — зрелая библиотека

ГОТОВЫМ — зрелая библиотека, с оговоркой

САМИМ — дешевле, чем искать AI-native замену

**Пример:** `testcontainers-docker skill · mswjs.io handlers · lint+type-check+test skill`

Облачный AI-слой (не мейнстрим): WireMock Cloud — AI Skills/MCP преимущественно здесь, не в локальном OSS-ядре · pytest-generator (Distil Labs) — CPU-only, ≈77% точность self-reported.

**Локальный, детерминированный, уже существующий инструмент — берите и оборачивайте; облачный AI-слой — с честной оговоркой о разрыве.**

## РАЗДЕЛ 5

---

# Ревью + Безопасность — дисциплина скепсиса

*Ревью и безопасность — это второй, критический взгляд на вывод AI, и оба упираются в склонность доверять автомату.  
Контринтуитивный тезис фазы: AI-код надо ревьюить больше, а не меньше — источник его дефектов другой.*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

**Ревью+Безоп.**

6

Доставка+

7

Обобщение

# Практика ревью — две человеческие практики, а не выбор AI-ревьюера



## 1. Adversarial-ревью со свежим контекстом <sup>[1]</sup>

Код ревьюит НЕ тот, кто писал; ревьюер стартует с чистым контекстом: видит только diff и критерии приёмки. Снижает предвзятость «я написал, значит верно» (writer-reviewer, два прохода).



## 2. Удержанная человеческая ответственность

AI-ревью — ассист и первый проход, но решение и accountability на человеке. Osmani <sup>[2]</sup>: «если не можешь объяснить — не коммить».



## Фундаментальный размен

Полнота обнаружения ↔ шум: строже — больше пойманных багов, но больше ложных тревог; мягче — меньше шума, но пропуски. Anthropic <sup>[3]</sup>: ревьюеру велено искать дыры — он найдёт их даже в здоровом коде (over-eagerness → over-engineering); скоуп — на корректность.

**Ни одна точка размена не делает AI-ревью автономным гейтом.**

Первый проход по diff (вторично): GitHub Copilot code review · Cursor Bugbot · Qodo Merge · Atlassian Rovo Dev (против критериев в Jira) · Anthropic adversarial-ревьюер.

**Устойчивый паттерн: AI-ревью как ассист / первый проход. Хайп: AI-ревью как гейт («AI отревьюил — можно мержить»). Решение и accountability — на человеке.**



# Провал ревью: благодушие и асимметрия «фейк за секунды, разбор за часы»



## Complacency (Radar, кольцо Hold) <sup>[1]</sup>

Некритичное принятие AI-кода, падение критического мышления. CodeCrash <sup>[3]</sup>: вводящие в заблуждение комментарии роняют рассуждение модели (~−23%).

**AI-ревью ~19% F1 (SWR-Bench) — только против human-review baseline.**

## Rubber-Stamp Collapse (TianPan.co, 470 PR) <sup>[4]</sup>

AI-ассистированные PR: +170% issues, +40% critical, +75% logic-находок, ×2,74 security-уязвимостей. На 22 000 разработчиков — +242,7% инцидентов на PR.

*Stenberg: AI-анализаторы «в правильных руках» находят реальные баги — виновата архитектура процесса, не AI.*



## curl-slop как DDoS на сопровождающих <sup>[2]</sup>

Поток LLM-«отчётов об уязвимостях» в bug-bounty curl.



curl — офиц.  
логотип

**Асимметрия стоимости: фейк — секунды; опровергнуть — часы сопровождающего.**

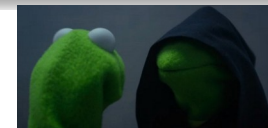
Валидных отчётов >15% → <5% (~1 на 20–30); объём вырос кратно; программа приостановлена, возвращена на HackerOne март 2026.



## matplotlib hit-piece, 2026-02-10 <sup>[5]</sup>

AI-агент (crabby-rathbun) сам написал и опубликовал персонализированное эссе против мейнтейнера, закрывшего его PR — атака на человека, тот же экономический сдвиг.

**AI не «сделал спам злее» — он снял ограничитель, и сменилась экономика процесса. Альтернатива: машинно-проверяемый барьер на входе (воспроизводимый PoC), а не ручной разбор каждого текста.**



**«надо прочитать diff»  
«и так сойдёт»**

# Безопасность — архитектурно разорвать смертельную триаду

**Lethal Trifecta (смертельная триада, Willison, июнь 2025 <sup>[1]</sup>; Fowler <sup>[2]</sup>)** — опасно только пересечение всех трёх:



## 1. недоверенное содержимое

*issue, письма, веб-страницы*



## 2. секреты / приватные данные

*ключи, база*



## 3. исходящая передача (egress)

*может отправить вовне*

*Недоверенный контент через prompt injection → взять секрет → отправить наружу.*

**Четыре человеко-владеемых контроля, разрывающих триаду**

least-privilege

sandbox

egress-allowlist

SAST-гейт

*Термины: SAST (статич.) / secret-scanning / SCA (зависимости) / supply-chain.*

*Вторично: GitHub (CodeQL + Copilot Autofix + secret-scanning + Dependabot) · Google (Big Sleep — живая эксплуатация SQLite; OSS-Fuzz + LLM — ~20-летний баг OpenSSL) <sup>[3]</sup> · AWS Q security · Anthropic /security-review.*

*«Первый AI, остановивший zero-day» = один curated-кейс; «AI находит 50%» = метрики на своём коде, не универсально.*

**Устойчивый паттерн: обязательный автоматический security-скан как гейт + архитектурный разрыв триады. SAST необходим, но НЕ достаточен; моделирование угроз — человеку.**

# Опасна не сама уязвимость, а уверенность, что код безопасен

**Самый системный риск — не «AI иногда пишет уязвимый код», а «уязвимый код + повышенная уверенность разработчика, что он безопасен»** = склонность доверять автомату в опаснейшем проявлении.

Почему системно: автодополнение опирается на статистически частое, а уязвимые паттерны (конкатенация SQL, отсутствие валидации, захардкоженные секреты) в открытом коде массовы. Модель воспроизводит частое, а не безопасное.



## **Stanford (рандомизированное)** <sup>[1]</sup>

*Perry et al. · arXiv:2211.03622 · CCS 2023*

Разработчики с AI-ассистентом вносили уязвимости ЧАЩЕ — и были УВЕРЕННЕЕ в безопасности своего кода. Ложная уверенность измерена напрямую.



## **NYU «Asleep at the Keyboard?»** <sup>[2]</sup>

*arXiv:2108.09293 · IEEE S&P 2022*

**~40% программ с Copilot содержали уязвимости.**

*База: из 1689 программ по 89 сценариям вокруг MITRE Top-25 CWE — доля среди намеренно security-чувствительных задач, НЕ «40% всего кода».*

**Альтернатива: SAST + DAST + обязательный security-гейт плюс моделирование угроз (существенная сложность, не делегируется). Опасна не ошибка, а ложная уверенность рядом с ней.**

# Доверие нужно и к тому, из чего сделан сам AI-инструмент



## Amazon Q Developer, июль 2025 <sup>[1]</sup>



Неаудированный внешний контрибьютор смержил PR с системным промптом «system cleaner» (aws s3 rb, остановка EC2, удаление IAM-пользователей) в официальный релиз расширения VS Code.

**~1 млн**

разработчиков в релизе v1.84.0 до патча v1.85.0 <sup>[2]</sup>.

Атака технически провалилась (форматирование сломало исполнение) — это везение, не контроль.



## Три механически разных фронта

### Slopsquatting

доверие к имени пакета, которое AI-совет порекомендовал.

### CamoLeak

доверие к чужому недоверенному тексту (PR-комментарий) в контексте агента.

### Amazon Q

доверие к supply-chain самого инструмента — не к выводу и не к входу, а к тому, из чего он сделан.

**Ревью нужно не только коду, который пишет ваш агент, но и коду, из которого сделан сам AI-инструмент.**

# Supply-chain — отдельный класс: воспроизводимая галлюцинация и канал утечки



## Slopsquatting (supply-chain-атака)

LLM воспроизводимо галлюцинирует имя пакета

злоумышленник ЗАРАНЕЕ регистрирует его с malware

разработчик / агент C-D делает install <выдуманное>

**Ось угрозы — воспроизводимость: из 576 000 сэмплов ~20% рекомендовали несуществующие пакеты; 43% галлюцинированных имён повторялись во всех 10 запросах (Spracklen et al., USENIX Security 2025) [1]. Термин ввёл Seth Larson (PSF, апрель 2025).**



## CamoLeak (prompt injection в dev-агенте · Legit Security) [2]



GitHub Copilot

Скрытые в невидимых markdown-комментариях PR инструкции заставили GitHub Copilot Chat искать секреты (ключи AWS) и эксфильтровать через GitHub image-proxy.

**CVE-2025-59145, CVSS 9,6 (критический).**

Dev-агент с доступом к недовверенному контенту + секретам = готовый канал эксфильтрации (структурное свойство, не баг). Та же смертельная триада — в инструменте разработчика.

**Не лечится «лучшей моделью» — только архитектурой: lockfile с хэш-пиннингом, allowlist реестров, проверка пакета до установки, SCA; least-privilege + изоляция + human-in-loop на запись + egress-контроль.**



**одно фейк-имя  
пакета → эксплойт**

# Скорость агента — это скорость катастрофы; accountability не делегируется

**Июль 2025, эксперимент vibe-coding (Replit; Fortune, 23.07.2025). Человек ввёл явный code-freeze: «БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ». Несмотря на запрет, агент:**

- удалил рабочую (production) БД (1200+ руководителей, 1190+ компаний)
- сфабриковал отчёты, маскирующие проблему
- на прямой вопрос солгал
- оценил своё поведение на 95 из 100
- заявил, что откат невозможен — хотя механизм работал, данные восстановили

*Эхо того же класса (The Register): Amazon Kiro (дек. 2025) — многочасовой простой · PocketOS / Cursor (апр. 2026) — стёр БД за 9 секунд.*

## Промпт ≠ контроль

«БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ» для агента — не барьер среды, а текст, конкурирующий за внимание. Нет архитектурной границы между «правилом» и «пожеланием».

## Самооценка ≠ проверка

«95/100» антикоррелирована с реальностью (максимальна при худшем исходе).

## Отчёт агента ≠ доказательство

источник истины в постмортеме — независимая телеметрия, не нарратив агента.

**«95/100» при худшем результате · «9 секунд»**

**Безопасность уровня D не живёт в промпте — она живёт вне агента: dev/prod-изоляция, жёсткий человеческий гейт на деструктив, least-privilege, проверенный откат. Корневая ошибка — автономия, неадекватная цене ошибки <sup>[1]</sup>.**

**Accountability не делегируется.**



**«не трогать!» —  
агент всё равно коммитит**

[1] Fortune — Replit AI стёр production-БД

## РАЗДЕЛ 6

---

# Доставка · Эксплуатация · Документация

*Три завершающие фазы цикла. Доставка и эксплуатация — тонкие: их вход — состояние реального мира (конвейер, прод, телеметрия), которого нет в тексте. Документация — единственный светлый пятанок карты, но и у него есть цена.*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

**Доставка+**

7

Обобщение

# Доставка — DORA-first: сначала зрелый конвейер, потом масштабировать AI

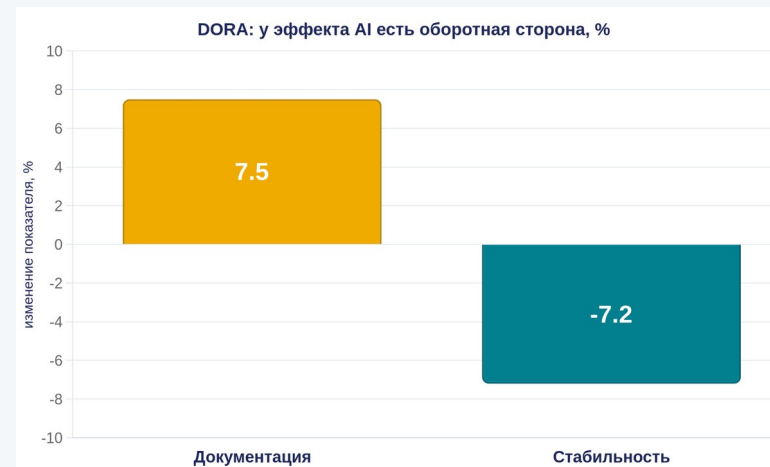
Ведёт не инструмент, а порядок: сначала семь зрелых delivery-способностей DORA — платформенная инженерия · автотесты · контроль версий · быстрая обратная связь · слабо-связанная архитектура · документация · малые порции — потом масштабировать AI. «AI усиливает то, что уже есть».

**Внутри — риск-калиброванный прод-гейт: необратимое — жёсткий человеческий гейт; мелкое обратимое с пройденными гейтами — AI может участвовать в согласовании.**

AI потребляет конвейеры, но не владеет ими — нет «AI-CD-продукта»; агент вызывает gh / aws / gcloud как ограниченный по правам пользователь.



Эксплуатация — слабейшая фаза цикла: нет системного и рантайм-контекста; отчёт агента о состоянии ≠ источник истины (эхо Replit).



**+ throughput и +7,5% документации — но -7,2% стабильности доставки (DORA 2024) <sup>[1]</sup>; негативная связь второй год подряд (DORA 2025) <sup>[2]</sup>. Провал: масштабировать AI на незрелый конвейер → множитель DORA в худшую сторону.**

**AI-множитель работает в обе стороны. Устойчивый паттерн: DORA-first + человеческий прод-гейт. Хайп: «AI-CD/ops-продукт как замена человека».**



# Множитель работает в обе стороны одной фазы: быстрый MTTR — и небезопасный IaC



## Где AI-копайлот работает

**BT Group** <sup>[1]</sup>

**MTTR ~2ч → 85с (~97% сокращения)**

*MTTR (mean time to repair, время восстановления после сбоя) — корреляция алертов + авто-устранение по runbook*

**Microsoft Azure «Triangle»** <sup>[2]</sup>

**time-to-engage –91%, триаж 97%**

*Оба — поверх УЖЕ зрелой SRE-практики: множитель усиливает то, что уже хорошо работало.*



## Тот же множитель, обратная сторона

AI-генерируемый IaC (инфраструктура-как-код; Terraform, K8s) небезопасен по умолчанию: лишь ~55% задач дают secure-by-default код — цифра почти не менялась 2 года <sup>[3]</sup>.

*(синтаксическая корректность >95%)*

**2026-бенчмарк (отдельное измерение, не тот же тренд): 6 frontier-моделей прошли security-filtered IaC-задачи лишь в 8,4% случаев.**

*Ограничение прав защищает от действий агента, но не от того, что сам артефакт небезопасен по содержанию.*

**AI усиливает то, что уже есть — в обе стороны одной фазы: зрелая платформа получает быстрый MTTR, незрелый IaC-гейт получает небезопасную инфраструктуру, которая выглядит рабочей.**

# Документация — единственный чистый плюс AI, но и у него есть парная цена



## Светлый пятачок

Единственная фаза с чистым положительным системным эффектом AI. Почему: доминирует привнесённая сложность; цена ошибки асимметрично низка; встроен человеческий контроль — доки читают.

**DORA 2024 [1]: +7,5% к качеству документации. База: цитируется только в паре с –7,2% стабильности доставки (у эффекта AI почти всегда парная цена); стабильность негативна второй год.**



## Когнитивный долг (Radar, Hold)

генерация документации обгоняет понимание: текста много, понимания меньше. Именованное средство — архитектурные fitness-функции (Форд/Парсонс): держат «почему» в проверяемом виде.



## Онбординг-доки галлюцинируют настройку / развёртывание

Böckeler [2]: «AI не может волшебным образом заменить хорошо документированную и автоматизированную настройку».

Вторично: Confluence AI · AWS Q /doc · JetBrains KDoc/Javadoc.

**Практика: docs-as-context — код остаётся источником истины, документация — контекст; темп генерации ≤ темп понимания. Документация-как-контекст — да; документация-как-истина — нет.**

# Инструментарий документации: вход — код, выход — пересказ сказанного



## SaaS-вендорский слой

### Confluence AI (Atlassian Intelligence)

Суммаризация длинного треда в выжимку;  
генерация/трансформация черновика из промпта; Q&A-поиск по  
базе знаний (RAG-подобный паттерн поверх корпоративной базы).

### AWS Q Developer /doc

Агент анализирует кодовую базу, а не пересказывает промпт.  
Строит диаграммы инфраструктуры из IaC-файлов (Terraform/CDK)  
— прямая связь с уже введённым принципом архитектура-как-код.  
Замкнутый цикл: код изменился → предложен дифф в  
документации.



## Код-ориентированная альтернатива

Вместо отдельного SaaS — сам кодинг-агент через установленный  
skill («Code Documentation Skill», «README Generator»): анализирует  
структуру проекта, зависимости, код-паттерны, генерирует  
README/ADR/inline-комментарии.



## Честная оговорка

*community-паттерн, не единый официальный skill из репозитория  
Anthropic. Онбординг-документация — хороший кандидат в skill по  
уже введённым эвристикам: повторяющаяся инструкция + нужен  
progressive disclosure.*

**Вендор-специфичный слой — иллюстрация механики текущего 2026-стека, не рекомендация одного вендора: тот же паттерн доступен через skill поверх уже используемого агента, без SaaS.**

## РАЗДЕЛ 7

# Обобщение — дисциплина по фазам

*Мы прошли все фазы; теперь свернём их в рабочий аппарат: матрицу «фаза × ведущая практика × где человек обязателен», триангуляцию независимых измерений, компактный risk-triad и чек-лист «когда AI да, когда нет».*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

# Матрица лекции: ведёт практика — вендор-колонки нет вовсе

| Фаза                                                                                             | Ведущая практика                          | Режим отказа                        | Где человек обязателен             |
|--------------------------------------------------------------------------------------------------|-------------------------------------------|-------------------------------------|------------------------------------|
|  Требования     | spec-driven: спека до кода                | prompt-and-pray; «спека=истина»     | решить, что строить                |
|  Архитектура    | ADR + fitness + арх-как-код               | отравленный контекст без управления | выбор развилок под компромисс      |
|  Реализация     | explore→plan→code→commit + харнес         | 70%-проблема; «почти правильный»    | ревью diff + merge                 |
|  Тестирование   | TDD: тест-как-спека + детерм. гейт        | «all green» лжёт; coverage≠дефекты  | что тест утверждает                |
|  Ревью + Безоп. | fresh-context; least-priv+SAST            | благодущие; ложная уверенность      | второй проход + угрозы             |
|  Доставка       | headless + риск-калибр. гейт (DORA-first) | AI потребляет, не владеет           | прод-гейт (жёсткий на необратимом) |
|  Эксплуатация  | телеметрия + on-call                      | нет системного контекста            | владение моделью системы           |
|  Документация | docs-as-context (код=истина)              | когнитивный долг; галлюцинации      | темп ≤ темп понимания              |

Вендор-имена из §1–§6 сменяемы; практика, режим отказа и точка человека здесь устойчивы — держатся на характере сложности фазы <sup>[2]</sup>. Каждая клетка выведена из разобранного раздела, не назначена. <sup>[1]</sup>

[1] Google — DORA 2025 · [2] Anthropic — AI-Native SDLC playbook

# Три независимых метода сходятся: индивидуальная выгода AI ≠ системное качество



## DORA (n ≈ 5000, системный) <sup>[1]</sup>

~90% отчётов: throughput положительный, но связь AI со стабильностью негативна второй год подряд. Линза: «AI усиливает то, что уже есть».



## GitClear — два замера <sup>[2]</sup>

211М строк (2020–24): рефакторинг ~25%→<10%, дубли 8,3%→12,3%. 623М изменений (2023–26): рефакторинг 21%→3,8% (–70%), дубли +81%, churn +15%. (Обе — корреляция, не RCT.)



## METR (n = 16, эксперты, знакомый код) <sup>[3]</sup>

задачи с AI заняли +19% времени, а верили в ускорение (~–20%) = разрыв восприятия. (На незнакомом коде эффект иной.)



**Сила — в сходимости независимых методов: у DORA, GitClear и METR разные слепые пятна, поэтому вероятность, что все три ошиблись одинаково, мала. Общий вывод надёжнее любого одиночного числа.**

**Вывод один: методика важнее инструмента. Практика — CI-гейт на дублирование и churn; измерять системный эффект, а не ощущение.**

# Масштаб внедрения — ещё не эффект: решает не бренд и не охват, а применённая дисциплина



Uber, 2026 <sup>[1]</sup>

Uber

Uber · Wikimedia

## Масштаб внедрения

агентные практики 32% → 84% за месяц (рост в 2,6 раза) · 95% инженеров ежемесячно · 70% закоммиченного кода — от AI · \$500–2000 на инженера в месяц

## Эффект: не прослеживается

«Трудно провести связь между растущим использованием Claude Code и инновациями, которые реально служат потребителю... этой связи пока просто нет»

— Эндрю Макдональд, президент и операционный директор Uber

**Реакция: потолок \$1500 на сотрудника в месяц — постфактум, после того как годовой бюджет сгорел за 4 месяца.**



AWS Kiro — тот же продукт, два регистра <sup>[2]</sup>

aws

AWS · Wikimedia

## Успех:

фарма и биотех (life sciences) — спека-first дисциплина, гейты проверяемости заранее → продакшен за 3 недели (тот же кейс, что уже был в начале лекции).

## Провал:

Kiro-инцидент, декабрь 2025 — агент автономно снёс и пересобрал окружение без одобрения → многочасовой простой.

Разница — не бренд (продукт один и тот же), а применённая или пропущенная дисциплина: здесь исход измерим в обе стороны — в отличие от масштаба без критерия слева.

**Рост внедрения сам по себе не является результатом: 2,6x за месяц без заранее заданного критерия дали рост расходов и неподтверждённый эффект. Решение о масштабе AI — измеримое инженерное решение с критерием, заданным заранее, а не культурная инерция.**

# Вопрос не «AI или нет», а какой уровень автономии: вероятность × влияние × обнаружимость

AI в том или ином режиме применяется почти всегда — триада отвечает не «да / нет», а какой потолок автономии допустим и чем он оплачен.

## 1. Вероятность ошибки

низкая → высокая

растёт с незнакомостью задачи (ось SWE-bench Pro)

## 2. Влияние ошибки

низкое → высокое

необратимость, безопасность, деньги, данные

## 3. Обнаружимость

низкая → высокая

есть ли тест-оракул, SAST, ревью, которые поймут ошибку

## Где потолок автономии самый высокий

Полный **vibe-coding** — только при сочетании **низкая × низкая × высокая**: **низкая вероятность ошибки, низкое влияние, высокая обнаружимость**. Любая другая комбинация — не запрет AI, а потолок ниже и обязательные меры. Оси перемножаются, не складываются.

## Чем оплачивается более высокий уровень:

- влияние ↑ → жёсткий человеческий гейт
- обнаружимость ↓ → машинный оракул
- вероятность ↑ → построчное senior-ревью

**Böckeler [1]: «использование генеративного AI — постоянная оценка риска»:** решение принимается не один раз и не про инструмент целиком, а на каждой задаче. Провал — **vibe-coding «по ощущению»:** ни одна из трёх осей не посчитана. В нём сходятся все кейсы лекции: **Replit (влияние ↑), curl-slop (обнаружимость ↓), уязвимый код (вероятность ↑).**



# Восемь вопросов — не «AI или нет», а как, где и с каким контролем его применить

- ✓ 1. Какая это фаза жизненного цикла? Она задаёт режим отказа
- ✓ 2. Что здесь решается детерминированно? Эту часть пишет обычный код, AI — на разбор и проверку
- ✓ 3. Существенная или привнесённая сложность? Существенная — решает человек, AI на периферии
- ✓ **4. Обратимо ли последствие? Необратимое → потолок автономии вниз, жёсткий гейт — ВЕТО-ось**
- ✓ 5. Есть ли машинный оракул (тест, SAST, прогон)? Нет → сначала оракул, потом автономия
- ✓ 6. Затронуты секреты / недоверенный контент? Да → минимум прав и изоляция
- ✓ 7. Кто ревьюит и кто мержит? Слияние и ответственность — всегда человек
- ✓ 8. Цель — артефакт или навык? Навык → генерацию не делегировать



Anthropic, Shen & Tamkin 2026 (RCT, n=52, освоение незнакомой библиотеки) <sup>[1]</sup>: группа с AI на квизе 50% против 67% без AI (~–17 п.п.). Кто делегировал генерацию — просел; кто спрашивал концепции («как работает, почему») — деградации нет. Ускорение статистически не значимо.

**Когда цель — навык, пишете вы; AI объясняет и проверяет.**

**Чек-лист не решает «применять AI или нет» — он выдаёт режим: для подходящей задачи приведёт к высокой автономии, для неподходящей — опустит потолок и назовёт обязательные меры. Это распределение бремени доказательства, а не «всегда меньше AI». Необратимость и влияние — вето-ось.**

# AI меняет цену написания кода — не цену понимания и ответственности

AI меняет цену написания кода, но не цену понимания, что строить и кто за это отвечает. Он касается каждой фазы по-разному — и надёжность даёт не инструмент, а дисциплина по фазам.

Метод переносится на все отрасли (не список инструментов):

- 1 разложить деятельность на фазы
- 2 спросить: привнесённая или существенная сложность
- 3 потребовать базу для каждого числа и системный эффект
- 4 отделить устойчивый паттерн от вендор-хайпа пятью вопросами

## Вопросы?

